Assume there are only two threads: `thread A (id == 0)` and `thread B (id == 1)`. Consider the following pseudo-code

```
static boolean flags = new boolean[2]; // initially zero

public void foo() {
    int i = ThreadId.get();                                    // F.1
    int j = 1 - i;                                             // F.2
    while (true) {                                             // F.3
        flags[i] = true;             // i would like to enter  // F.4
        if (flags[j] == false) {     // you don't              // F.5
            if (flags[i] == true) {  // my request was not reset // F.6
                break;               // i win                  // F.7
            }
        } else {
            // looks like we have a contention
            flags[i] = false; // retreat                       // F.8
            flags[j] = false; // forcibly reset competitor      // F.9
        }
    }
}
```

- Is it wait-free?
  No. Example: `B` constantly resetting `A`'s request.
- Is it lock-free?
  No. Step by step execution and ultimate death on the `F.4-6`, both threads always taking the `else` branch. None progress.
- Is it obstruction-free?
  Lets assume that thread `j` does not take more steps. `j` could have froze with its flag set, or not set.
  - if set: `i` unsets, breaks on the next iteration.
  - if unset: then if `i` is set it just breaks, if `i` is unset, it sets itself on the next iteration and breaks.
- Does it guarantee starvation-freedom?
  No. `A` waits. Thread `B` can repeatedly enter the lock, unset `A`'s flag, set its own flag and break.
- Does it guarantee deadlock-freedom?
  Both threads are always doing something. Even if they depend on each other to leave foo, they dont ultimately wait for something being done by the other thread. There is no blocking, threads just do something infinitely.